



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

并行程序设计实验报告

SIMD 编程实验

姓名：吕昊远

学号：2410668

专业：计算机科学与技术

指导教师：王刚

2026 年 5 月 10 日

目录

一、 实验背景	1
二、 实验原理	1
(一) 问题定义	1
(二) SIMD 并行优化	1
(三) SQ (Scalar Quantization - 标量量化)	2
(四) PQ (Product Quantization - 乘积量化)	2
(五) PQ + Rerank 以及其他优化	3
三、 程序设计与实现	4
(一) Flat 暴力算法	4
(二) Flat - SIMD	4
(三) SQ - SIMD	5
(四) PQ - SIMD	7
(五) PQ - Rerank - SIMD	8
(六) PQ - Final	9
四、 实验结果分析	10
(一) 数据总览	10
(二) 参数权衡	13
(三) perf 分析	13
(四) 问题探讨	14
五、 总结与体会	14
六、 附录：代码仓库链接	15

一、实验背景

在人工智能领域中，向量搜索是各类模型的基础。最近邻搜索（NNS），主要用于在高维向量空间中查询向量相似度最高的向量。依托深度学习技术，图像、文本、音频等各类数据可转化为具备语义关联特性的高维向量，数据的语义相似程度通过在向量空间的距离直观体现，这也让最近邻搜索技术广泛适配于推荐系统、搜索引擎及大语言模型等主流应用场景。

在实际场景中，待检索的向量数据量级往往大于千万级别，且向量维度也普遍在 100 维以上。传统精确搜索算法复杂度高、检索耗时久，难以满足现实中的检索需求。为了有效平衡检索精度与计算效率，近似最近邻搜索（ANNS）成为高维向量检索的优选思路，在保障检索精度的前提下大幅缩减计算耗时。在本次实验中，将围绕近似最近邻搜索算法开展相关的实验与分析。

二、实验原理

（一）问题定义

- 给定数据集 $X = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\} \subset \mathbb{R}^d$ 。
- 对于查询向量 $\mathbf{q} \in \mathbb{R}^d$ ，ANNS 的目标是找到集合 $S \subseteq X$ ，使得 S 中的向量与 $\mathbf{q}$ 足够接近，并满足一定的准确度要求。

准确度衡量：常用召回率（recall）评估 ANNS 的准确性：

$$\text{recall}@k = \frac{|KNN(\mathbf{q}) \cap KANN(\mathbf{q})|}{k}$$

其中 $KNN(\mathbf{q})$ 是精确的 k 个最近邻集合， $KANN(\mathbf{q})$ 是 ANNS 返回的 k 个近似最近邻集合。本实验中，我们关注在达到特定召回率（0.9/0.95）时的算法延迟。

- 优化方向：在本实验中，主要通过以下几个方面对 ANN 算法进行优化：
 - Flat - SIMD
 - SQ - SIMD
 - SQ - SIMD

接下来将对这几种思路的算法原理进行详细讲解。

（二）SIMD 并行优化

原理： SIMD(单指令多数据) 通过在单个指令下同时处理多个数据来实现计算效率的提升。在本实验的 ANN 选题中，SIMD 主要用于对距离计算进行并行优化，从而提升算法的搜索性能。

数学公式：

- **欧几里得距离：**对于两个向量 $\mathbf{x}$ 和 $\mathbf{y}$ ，其欧几里得距离定义为：

$$\delta(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^d (x_i - y_i)^2}$$

- **内积距离：**内积距离常用于余弦相似度计算，定义为：

$$\delta(\mathbf{x}, \mathbf{y}) = 1 - \sum_{i=1}^d x_i \cdot y_i$$

在本实验中，主要考虑内积距离。SIMD 并行计算多个维度的乘积和累加，从而实现内积运算的加速。通过 SIMD 指令集，如 SSE/AVX (x86) 与 Neon (ARM)，我们可以将向量数据打包到寄存器中，并行执行各种运算操作。这一优化思路将作为本实验的核心与基础，会在后续的各种优化算法中体现。

(三) SQ (Scalar Quantization - 标量量化)

SQ 标量量化是面向高维向量检索的轻量化数据压缩与近似计算技术，其核心在于：在尽可能保留向量的相似关系的前提下，将高精度浮点向量转换为整型向量，从而降低内存占用、减少内存带宽压力，来提升检索效率。本实验采用无偏移对称标量量化的方法，将 32 位浮点向量压缩为 8 位整型向量，量化过程主要遵循以下原则：

- 全局统一量化：全库向量使用相同的量化尺度，保证向量间的相对距离关系不变；
- 对称量化：不引入偏移量，确保量化后向量的内积排序与原始浮点向量一致，最大限度保留检索精度。

在此基础上，根据指导书要求，还可以进行重排以提升召回率。标量量化本质上是对浮点向量的近似压缩，会引入微小误差。如果直接用量化后的距离计算 Top-k，很可能把真正的近邻的漏掉，导致召回率下降。为了解决该问题，使用重排设计，主要分为以下两个阶段：

- 粗排阶段（量化空间）：先用量化后的 int8 向量快速计算距离，选 Top-p 个候选向量 ($p > k$)。这一步速度很快，但其得出的结果是近似的；
- 精排阶段（原始空间）：找出这 p 个候选向量的原始浮点数，用 Flat-SIMD 的算法重新计算精确距离，再选出最终的 Top-k。这一步只需要处理少量候选，开销很小，但能大幅提升召回率，保障检索精度。

这种设计的好处在于，可以有效利用优化算法的效率优势，又用精排减少了量化的误差，维持了召回率的稳定，使得我们在实际应用中可以更加灵活地 trade-off。

注：在 SQ 这部分，由于采取的压缩策略对于向量间关系的维护，并未损失多少精度，召回率已经较高。对其进行重排优化后效果并不明显，其重要性主要体现在接下来的 PQ 算法上，后续将会在实现部分详细讲解。

(四) PQ (Product Quantization - 乘积量化)

PQ 乘积量化主要引入了聚类这一算法，其核心思路为：将高维向量切分成多个低维子向量，对每个子向量空间独立聚类量化，用聚类质心的编号代替原始子向量存储；这样处理后，查询时可以通过查表法快速计算近似距离，将时间复杂度从 $O(D)$ 降至 $O(M)$ (M 为子空间数量， $M \ll D$)。

在离线量化的预处理阶段，首先对向量进行切分，将所有 N 个原始 D 维基向量，均匀切分为 M 个独立的低维子向量，然后对每一个子空间分别执行 K-Means 聚类，得到该子空间的 K 个质心。对每个基向量的每个子向量，找到距离最近的质心，记录其编号 ($0 \sim K-1$)；最终每个基向量被压缩为长度为 M 的编号序列，即 PQ 编码。

接下来的搜索阶段则是采用了非对称距离计算 (ADC) 的核心方法：PQ 并不直接计算压缩基向量与查询向量的距离，而是预计算查询向量与所有质心的距离，在搜索时查表求和得到近似距离。流程中的核心主要在于以下两部分：

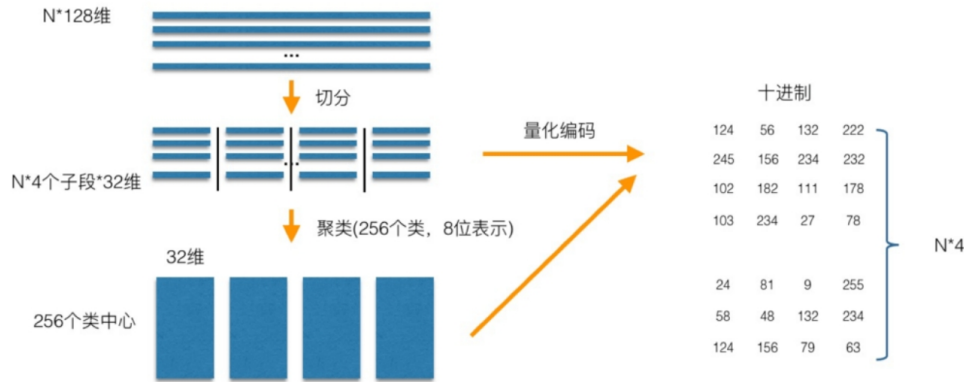


图 1: PQ 示例

- 构建查找表 LUT：对查询向量 Q 做相同切分： $Q \rightarrow [Q_1, Q_2, \dots, Q_M]$ ，预计算每个子查询向量 Q_m 与第 m 个子空间 K 个质心的距离，存入 $LUT[m][c]$
- 近似距离计算：遍历所有基向量的 PQ 编码，对于每个基向量 i ，其编码为 $[c_1, c_2, \dots, c_M]$ ，近似距离 $= LUT[1][c_1] + LUT[2][c_2] + \dots + LUT[M][c_M]$ ，通过这样的近似，我们可以直接查表求和，无需逐维度计算，大大提升了计算的性能。

尽管从性能角度来看，PQ 算法的效率在理论上具备极大的优势，但在具体实验中，PQ 往往出现召回率过低的问题。从量化误差到近似查表，单纯的 PQ 算法存在严重的量化噪声，一旦子空间数 M 较小，召回率会非常低，这实际上是算法自身机制带来的缺陷。尽管我们可以通过提升 M 来缓解，但是其效果相对有限，我们需要一种有效的方式来提升检索的精度。

(五) PQ + Rerank 以及其他优化

为了彻底解决召回率的问题，我们可以将 PQ 与重排结合起来，通过重排来弥补 PQ 损失的精度。其具体实现思路和上一小节中 SQ+ 重排的思路基本一致。通过与重排相结合，其召回率通常可以从 0.2 0.4 提升至 0.9，完美解决 PQ 算法在实际应用中“快而不准”的问题。

然而，我们可以发现，实际上在重排阶段还有很多的优化空间。主要有以下几个方面：

• 访存模式优化：AoS $\rightarrow$ SoA

在基础的 PQ 算法中，编码数据通常是按向量来组织的（AoS），这在计算 ADC 时会导致内存访问在多个子空间之间跳转，不利于缓存预取。

我们可以将其重构为 SoA 布局：将库中所有向量的第 m 个子空间的编码存储在连续的内存块中。在粗排扫描时，CPU 对编码表的读取变成了完全的线性连续访问，这将充分利用 CPU 的硬件预取器，有利于降低 Cache Miss，提升有效访存带宽。

• LUT 构建：跨中心点并行

在查找表的构建这部分，我们可以利用 SIMD 实现跨中心点的并行化：利用 `vld1q_f32` 加载查询向量的子段，并使用 `vdupq_n_f32` 广播查询向量单一维度的值，使其同时与 4 个（或更多）中心点的对应维度值相乘。理论上，LUT 的构建速度将会提升 4 倍，在高量化精度场景下，这一优化可以显著降低查询前的准备耗时。

• 局部重排并行

在重排阶段，算法依赖于一个全局优先级队列来筛选候选集，频繁的线程交互带来了高昂的锁竞争。我们可以考虑通过把重排这部分局部化，每条线程都有自己独立的局部粗排队列和局部精排队列。线程在完成其负责的扫描后，立即在线程内触发精排。整个搜索过程无需任何原子操作或互斥锁，仅在最终阶段进行一次小规模的结果归并。这种设计进一步消除了锁开销，充分发挥了多核性能。

三、 程序设计与实现

(一) Flat 暴力算法

在实验框架中，已经给出了最基础的 Flat 算法，首先依照框架要求使用 test.sh 进行测试，得到以下结果：

average recall: 0.99995

average latency: 19024.6us

该算法直接进行暴力遍历，因而耗时较长，但在召回率上较为严格。在运行过程中，存在大量的计算可以进行并行化处理，同时也存在搜索策略上的改进。我们以未进行任何优化的算法的测试数据作为基准，接下来在此基础上，进行各方面的探索与优化改进。

(二) Flat - SIMD

在 Flat-SIMD 这部分，算法不进行策略上的改进，仅依靠 SIMD 进行并行优化。针对 ARM NEON 架构，主要使用以下方法：

- **并行计算**：利用 vld1q_f32 一次性将 4 个浮点数载入寄存器，并配合 vfmaq_f32 执行乘加运算，将原本需要多次循环的标量计算缩减为单次指令执行；
- **结果合并**：原始的累加操作改为 vaddvq_f32，将并行计算的 4 个中间结果一次性求和，提升累加效率。

Algorithm 1 Flat-SIMD 伪代码

Input: 查询向量 $Query$, 基础库 $Base[N]$, 维度 D , 返回数量 k

Output: Top- k 结果队列 q

```

1: 初始化优先级队列  $q$  (大小为  $k$ )
2: for  $i = 0$  to  $N - 1$  do
3:    $sum\_vec \leftarrow [0.0, 0.0, 0.0, 0.0]$  ▷ 128 位 SIMD 寄存器
4:   for  $d = 0$  to  $D - 1$  step 4 do
5:      $q\_vec \leftarrow Load(Query + d)$ 
6:      $b\_vec \leftarrow Load(Base[i] + d)$ 
7:      $sum\_vec \leftarrow MultiplyAdd(sum\_vec, q\_vec, b\_vec)$  ▷ SIMD 向量化乘加 (FMA)
8:   end for
9:    $distance \leftarrow 1.0 - SumElements(sum\_vec)$  ▷ 处理尾部并转为距离
10:  if  $distance < q.top()$  or  $q.size() < k$  then
11:     $q.push(distance, i)$ 
12:    if  $q.size() > k$  then
13:       $q.pop()$ 
14:    end if
15:  end if

```

16: **end for**

17: **return** q

在这部分，我们仅对 Flat 算法进行了基础的 SIMD 优化，主要验证代码思路的可行性，并为后续的进一步优化奠定基础。理论上如上算法的加速比应该趋近于 4，是一种简单而有效的基准优化方法。同时，由于未对搜索策略进行更改，其召回率也和纯暴力算法一致，保持在 0.99995 的严谨水平。

在实验中，也考虑使用多寄存器展开，但是效果不佳，并未有实质性的提升。对于这一现象，经过分析后，猜测可能是实验框架 test.sh 内置的编译采用了较高版本的编译器，其通常自带编译优化。使用弱优化/无优化的编译指令重新进行实验。

bash

```
1 g++ main.cc -o main -O1 -fno-tree-vectorize -march=armv8-a -ffast-math
```

弱优化编译下，测量纯暴力算法得到结果 average latency : 20981.2us，慢于原实验框架下的测试结果。重新计算加速比可知，SIMD 优化相较于该算法的实际加速比达到 3.88，已经接近于 SIMD 优化的极限。对其进行其余的优化反而会带来额外开销，加之框架本身自带编译优化，SIMD 的优化空间一部分被其余开销抵消，因而效果不是很明显。

在这里，考虑到其余算法，实验的基准结果仍采取实验框架下的测试数据。此处手动编译仅作为一例现象探讨，后续的加速比等分析仍以框架下的测试结果为准。

(三) SQ - SIMD

SQ 实现通过量化将浮点值映射到 8 位无符号整数，在这部分，除了实现 SQ 的逻辑之外，还需要和 SIMD 结合起来。

Algorithm 2 SQ-SIMD 伪代码

Input: 查询向量 $Query$, 量化基础库 $SQ_Base[N]$ (int8), 维度 D , 返回数量 k

Output: Top- k 结果队列 q

```
1: 初始化优先级队列  $q$ 
2:  $max\_val \leftarrow$  基础库最大绝对值
3:  $scale \leftarrow 127.0 / max\_val$ 
4:  $Query\_i8 \leftarrow Quantize(Query, scale)$  ▷ 将查询向量转为 int8
5: for  $i = 0$  to  $N - 1$  do
6:    $sum\_vec\_32 \leftarrow [0, 0, 0, 0]$  ▷ 32 位整型累加寄存器，防溢出
7:   for  $d = 0$  to  $D - 1$  step 16 do
8:      $q\_vec\_8 \leftarrow Load\_16x8(Query\_i8 + d)$  ▷ 一次处理 16 个 int8 数据
9:      $b\_vec\_8 \leftarrow Load\_16x8(SQ\_Base[i] + d)$ 
10:     $prod\_low \leftarrow Multiply(LowHalf(q\_vec\_8), LowHalf(b\_vec\_8))$  ▷ 拆分高低位
11:     $prod\_high \leftarrow Multiply(HighHalf(q\_vec\_8), HighHalf(b\_vec\_8))$ 
12:     $sum\_vec\_32 \leftarrow Accumulate(sum\_vec\_32, prod\_low, prod\_high)$ 
13:  end for
14:  $distance \leftarrow 1.0 - (SumElements(sum\_vec\_32) / (scale \times scale))$  ▷ 反量化还原近似浮点距离
15:  if  $distance < q.top()$  or  $q.size() < k$  then
16:     $q.push(distance, i)$ 
17:  if  $q.size() > k$  then
```

```

18:         q.pop()
19:     end if
20: end if
21: end for
22: return q

```

在具体实现上，主要分为离线预处理与在线搜索两个较为独立的执行阶段。离线预处理在检索前进行，仅需执行一次，核心目标是将原始 32 位浮点基向量转换为 8 位整型向量，完成数据压缩与量化参数计算。程序会先为量化后的整型向量分配内存空间，其大小为原始向量库的四分之一，随后遍历所有浮点向量元素，计算出全局最大绝对值，以此为依据确定量化缩放因子，再通过多线程并行的方式，将每个浮点元素与缩放因子相乘并截断为 int8 整型，最终得到压缩后的整型基向量库。

预处理函数

```

1 void offline_sq(float* base, size_t base_number, size_t vecdim) {
2     sq_base = new int8_t[base_number * vecdim];
3     float max_val = 0;
4     for (size_t i = 0; i < base_number * vecdim; ++i) max_val = std::max(
5         max_val, std::abs(base[i]));
6     sq_scale = 127.0f / max_val; // 对称量化
7     #pragma omp parallel for
8     for (size_t i = 0; i < base_number * vecdim; ++i) {
9         sq_base[i] = (int8_t)(base[i] * sq_scale);
10    }

```

这里的量化过程仅通过统一的缩放因子将原始的浮点数映射到 int8 类型的 $[-127, 127]$ 取值区间，不添加任何偏移量调整数值分布。这种量化方式能够保证向量在量化前后，内积的相对大小与排序关系完全保持一致，从根源上尽可能避免的检索排名的误差。

在在线搜索阶段，其逻辑与 Flat 较为相似，区别在于需要先将查询向量也使用同一规则进行压缩，计算完成后进行反量化还原，除以缩放因子的平方得到浮点近似内积，结果处理后再使用优先队列维护，最终输出 Top-K 检索结果。

在 SIMD 的部分，使用 16 个压缩后的 int8 元素为步长执行操作，单次指令可同时加载 16 个基向量分量与 16 个查询向量分量，先将其拆分高低 8 个分量完成乘法运算，为避免溢出，将 int8 类型的乘积结果扩展为 int16 类型，再累加至 int32 寄存器中完成并行乘加累加。相比于 Flat-SIMD 的 4 路浮点并行，SQ-SIMD 实现了 16 路整型并行计算，同时全程采用连续内存加载方式，充分发挥了 SIMD 的并行优势，大幅提升了内积计算的吞吐量与内存访问效率。

在实验指导书中，还提到了重排，这部分也对其进行了相关的实现。仅需在代码计算 Top-K 的部分改为计算 Top-K' ($K' > K$)，最后再对这 K 个候选向量进行小规模的精排，即可得到最终的结果。结合进行对比测试，得到如下结果：

在测试场景中，仅依靠基础的 SQ+SIMD 就已经能实现较高的召回率。当 rerank_k 为 11 或 12 时（相对于 $k = 10$ 仅进行极小的扩容），召回率已经体现出极高的精度，而当其大于 15 的时候，召回率已接近极限。尽管在 SQ 效果良好的情况下，重排的优化效果并不显著，但在这里我们通过实践验证了其可行性与有效性，为后续的 PQ 与重排结合奠定了基础。

表 1: SQ 测试结果

rerank_k	平均召回率	平均延迟 (μs)
/	0.963406	2795.77
11	0.986502	2855.21
12	0.995051	2879.86
15	0.99955	2898.41
20	0.99995	2947.29

(四) PQ - SIMD

PQ 主要实现将高维向量分解为子向量, 然后单独进行量化, 再通过查表来加速距离计算。与 SQ 类似, PQ 也分为离线和在线两部分。离线阶段完成乘积量化的编码与质心生成, 在线阶段通过查找表实现快速近似检索, 同时依托 SIMD 指令完成核心计算的并行加速。

离线部分主要完成向量的量化压缩。程序逐个子空间进行处理, 先提取所有基向量对应位置的子向量数据, 通过聚类得到每个子空间的质心, 再为每个向量的子向量匹配内积最大的质心, 将其编号作为 PQ 编码存储。最终所有高维浮点向量都被压缩为长度为 16 的 `uint8_t` 编码序列, 从而完成 Codes 编码表和 Centroids 质心表的构建。

Algorithm 3 PQ-SIMD

Input: 查询 *Query*, PQ 编码表 *Codes*[*N*][*M*], 质心 *Centroids*[*M*][*K*][*D*/*M*], 返回数量 *k*

Output: Top-*k* 结果队列 *q*

```

1:                                     ▷ 阶段 1: 构建查找表 (LUT)
2: 初始化 LUT[M][K]
3: for m = 0 to M - 1 do
4:   for c = 0 to K - 1 do
5:     LUT[m][c] ← SIMD_InnerProduct(Query[m], Centroids[m][c])
6:   end for
7: end for
8:                                     ▷ 阶段 2: ADC 查表计算距离
9: 初始化优先队列 q
10: for i = 0 to N - 1 do
11:   ip ← 0.0
12:   for m = 0 to M - 1 do
13:     ip ← ip + LUT[m][Codes[i][m]]                                     ▷ Gather 操作
14:   end for
15:   distance ← 1.0 - ip
16:   if distance < q.top() or q.size() < k then
17:     q.push(distance, i)
18:     if q.size() > k then
19:       q.pop()
20:     end if
21:   end if
22: end for
23: return q

```

在线搜索部分核心采用非对称距离计算完成近似检索。首先构建查找表 LUT，程序遍历子空间和质心，计算查询向量的子向量与每个质心的内积，将结果存入二维数组 LUT 中，完成所有预计算工作；接下来通过查表来计算距离，读取其 PQ 编码后直接查表累加得到近似内积，再转换为余弦距离。输出阶段逻辑类似，通过优先队列维护得到最终结果。

然而，在实际测试过程中，我们发现其召回率出现断崖式下跌（ $M=4$ 时，召回率仅为 **0.11**）。单纯依靠 PQ 进行量化近似存在很大的误差，虽然其速度显著加快，但是其召回率过低，精度损失过多。通过算法的原理分析，我们尝试提升子空间的数量 M 来缓解量化时带来的误差，但是其效果依然有限。即使将 M 提升至 32，召回率也不到 0.5，且其边际效应愈发明显。

表 2: PQ-SIMD 测试结果

M	平均召回率	平均延迟 (us)
4	0.11475	554.338
8	0.2543	1084.04
16	0.433	2365.27
32	0.4973	4562.5

实际上我们现在可以知道，单纯的 PQ 真正的优势在于其极快的速度，我们完全可以牺牲一部分性能，通过其他策略维护其召回率的稳定。在刚刚的 SQ 部分，我们已经实现了重排这一算法，将重排的理念应用于 PQ，可以大幅提升算法的召回率，而 PQ 自身的效率优势也为重排带来了较为宽裕的取舍的空间。

(五) PQ - Rerank - SIMD

在这部分，我们将 PQ 与重排相结合，着重解决 PQ 召回率过低的问题。与 SQ 类似，将基础的 PQ 作为粗排阶段，选出候选的 Top-K'，然后在 K' 个候选向量中进行较小规模的精排，即可得到最终的结果。

Algorithm 4 PQ-Rerank-SIMD

Input: 查询 $Query$, 库 $Base[N]$, $Codes[N][M]$, $Centroids$, 数量 k , 粗排数量 $rerank_k$

Output: Top- k 结果队列 $fine_q$

```

1:  $LUT[M][K] \leftarrow \text{Build\_LUT\_SIMD}(Query, Centroids)$ 
2:
3: 初始化队列  $coarse\_q$  (大小为  $rerank\_k$ )
4: for  $i = 0$  to  $N - 1$  do
5:    $ip \leftarrow \sum_{m=0}^{M-1} LUT[m][Codes[i][m]]$ 
6:    $distance \leftarrow 1.0 - ip$ 
7:   if  $distance < coarse\_q.top()$  or  $coarse\_q.size() < rerank\_k$  then
8:      $coarse\_q.push(distance, i)$ 
9:     if  $coarse\_q.size() > rerank\_k$  then
10:       $coarse\_q.pop()$ 
11:   end if
12: end if
```

▷ 阶段 1: ADC 粗排

```

13: end for
14:
15: 初始化最终队列 fine_q (大小为 k)
16: while coarse_q is not empty do
17:   idx  $\leftarrow$  coarse_q.pop().index
18:   exact_ip  $\leftarrow$  SIMD_InnerProduct(Query, Base[idx]) ▷ 读取原始 float 向量
19:   exact_distance  $\leftarrow$  1.0 - exact_ip
20:   if exact_distance < fine_q.top() or fine_q.size() < k then
21:     fine_q.push(exact_distance, idx)
22:     if fine_q.size() > k then
23:       fine_q.pop()
24:     end if
25:   end if
26: end while
27: return fine_q

```

经过重排后，在 $M = 8$, $\text{rerank_k} = 500$ 的参数下进行测试，其召回率高达 0.944，而延迟为 1560.89us，相较于不使用重排 (1084.04) 并未增加过多。经过测试，我们验证了重排对于精度提升的可行性与高效性，仅通过少量延迟的增加，即可让召回率获得大幅提升。经过重排优化后的 PQ 算法终于具备了解决实际问题的能力，其在效率上的优势得以体现。

(六) PQ - Final

截至到这里，我们终于实现了完整有效的 PQ+ 重排 +SIMD 这一混合算法。重新审视这一流程，我们可以发现，其中还有许多可以进行优化的地方。在这部分，将对算法进行深度优化，使其效率得到进一步的提升。

Algorithm 5 PQ-Final

Input: 查询 *Query*, *Base*[*N*], SoA 编码表 *Codes_SoA*[*M*][*N*], SoA 质心 *Cents_SoA*[*M*][*D/M*][*K*], *k*, *rerank_k*

Output: 最终 Top-*k* 队列 *final_q*

```

1:
2: for m = 0 to M - 1 do
3:   for c = 0 to K - 1 step 4 do
4:     acc_ip  $\leftarrow$  [0.0, 0.0, 0.0, 0.0]
5:     for d = 0 to SubDim - 1 do
6:       q_val  $\leftarrow$  Broadcast(Query[m][d]) ▷ 广播查询维度
7:       c_vals  $\leftarrow$  Load_4_Centroids(Cents_SoA[m][d][c...c + 3])
8:       acc_ip  $\leftarrow$  MultiplyAdd(acc_ip, q_val, c_vals)
9:     end for
10:    Store(LUT[m][c...c + 3], acc_ip)
11:  end for
12: end for
13:
14: 声明全局结果集 ThreadResults[NumThreads]
15: Parallel Execution (OpenMP):

```

```

16:   $tid \leftarrow \text{GetThreadID}()$ 
17:  初始化局部队列  $local\_coarse\_q$  (大小  $rerank\_k$ ),  $local\_fine\_q$  (大小  $k$ )
18:                                     ▷ 跨向量连续访问 (SoA Cache 友好)
19:  for  $i \in$  线程分配的任务块 do
20:       $ip \leftarrow 0.0$ 
21:      for  $m = 0$  to  $M - 1$  do                                     ▷ 此处已循环展开
22:           $ip \leftarrow ip + LUT[m][Codes\_SoA[m][i]]$              ▷ 内存连续
23:      end for
24:      根据  $1.0 - ip$  维护更新  $local\_coarse\_q$ 
25:  end for
26:                                     ▷ 线程内精排 (AoS 布局重计算)
27:  while  $local\_coarse\_q$  is not empty do
28:       $idx \leftarrow local\_coarse\_q.pop().index$ 
29:       $exact\_distance \leftarrow 1.0 - \text{SIMD\_InnerProduct}(Query, Base[idx])$ 
30:      根据  $exact\_distance$  维护更新  $local\_fine\_q$ 
31:  end while
32:   $ThreadResults[tid] \leftarrow local\_fine\_q$ 
33:                                     ▷ 阶段 3: 全局归并
34:   $final\_q \leftarrow \text{MergeAll}(ThreadResults)$ 
35:  return  $final\_q$ 

```

为了进一步提升算法性能，我们针对计算机底层的访存特性与指令流水线进行了重构。传统 PQ 算法通常采用按向量组织的布局，这在查表计算非对称距离时会导致内存的跨步访问；这里将其改为 SoA (Structure of Arrays) 布局，使得粗排扫描阶段的内存读取变成完全的线性连续访问。这一改动充分发挥了 CPU 的硬件预取机制，可以显著降低缓存缺失率。同时，在构建查找表时，还利用 SIMD 的广播指令实现了跨质心的向量化并行，一次性将单个查询维度与多个质心同时计算，将预处理阶段的耗时也进行了优化缩减。

此外，锁竞争带来的延迟也可以通过架构的改进进行优化。在每个 OpenMP 线程处理分配到的连续数据块时，改为独立维护局部的粗排与精排队列，将流水线先放在每个线程内部，然后在所有线程执行完毕后再进行一次小规模的全局归并。这种局部重排机制不仅在消除了多线程的同步开销，实现了接近线性的多核加速比，还在统计特性上带来了一定的优化，算法强制从不同数据分块中提取候选者，可以有效避免量化噪声导致的局部严重失真问题（大量极其近似的向量阻塞队列）。

四、实验结果分析

（一）数据总览

本实验使用 DEEP100K 数据集，依照服务器上的实验框架，实现并测试不同的 SIMD 优化方法在 ANN 问题中的性能表现。以下为主要的测试结果。

表 3: 测试结果

算法名称	M	rerank_k	平均召回率	平均延迟 (us)	加速比
Flat	-	-	0.99995	19024.6	1.00
Flat-SIMD	-	-	0.99995	5404.35	3.52
SQ-SIMD	-	-	0.963406	2795.77	6.80
PQ-SIMD	4	-	0.11475	554.338	34.32
PQ-SIMD	8	-	0.2543	1084.04	17.55
PQ-SIMD	16	-	0.433	2365.27	8.04
PQ-SIMD	32	-	0.4973	4562.5	4.17
PQ-Rerank-SIMD	4	500	0.796402	998.589	19.05
PQ-Rerank-SIMD	8	500	0.944404	1560.89	12.19
PQ-Rerank-SIMD	16	500	0.994351	2967.29	6.41
PQ-Rerank-SIMD	8	250	0.872104	1325.93	14.35
PQ-Rerank-SIMD	8	1000	0.980252	2090.26	9.10
PQ-final	8	125	0.980002	1114.09	17.08
PQ-final	8	250	0.994851	1295.68	14.68
PQ-final	4	125	0.901604	1023.16	18.59
PQ-final	4	250	0.964603	1214.9	15.66
PQ-final	16	125	0.99885	1453.95	13.08
PQ-final	16	250	0.9998	1670.62	11.39
PQ-final	8	25	0.831704	1008.9	18.86
PQ-final	16	25	0.965454	1349.62	14.10
PQ-final	32	25	0.985552	2402.67	7.92

- Flat-SIMD 依托于暴力算法，虽然保证了准确的召回率 (>0.999)，但由于需要进行全库的精确内积计算，其耗时在所有策略中最高。
- SQ-SIMD 在牺牲小部分精度的前提下，有效降低了内存带宽压力，使延迟相较于暴力搜索策略有了显著提升。同时可以也结合重排策略，以满足高精度场景下的需求
- PQ-SIMD 体现出极致的效率优势，但算法的量化误差较大，导致召回率断崖式下跌；随着子空间数量 (M) 的增加，召回率虽有改善，但仍远远无法到达要求的精度，且带来的延迟也在不断增加。
- PQ-Rerank-SIMD 有效弥补了 PQ 损失的精度，随着候选集大小 (rerank_k) 的增大，召

回率可恢复至 0.95 以上的较高水平，但这也伴随着显著增加的计算延迟作为代价。

- PQ-Final 使用局部精排的无锁化结构，利用分布式采样的优势与 Cache 友好的硬件架构设计，在达到同等极高召回率（0.98）的前提下，能够将延迟压缩至 PQ-Rerank-SIMD 的一半左右，是本实验中优化的最终模型，实现了性能与精度的最优解。

对部分测量数据进行可视化处理，得到以下图象：

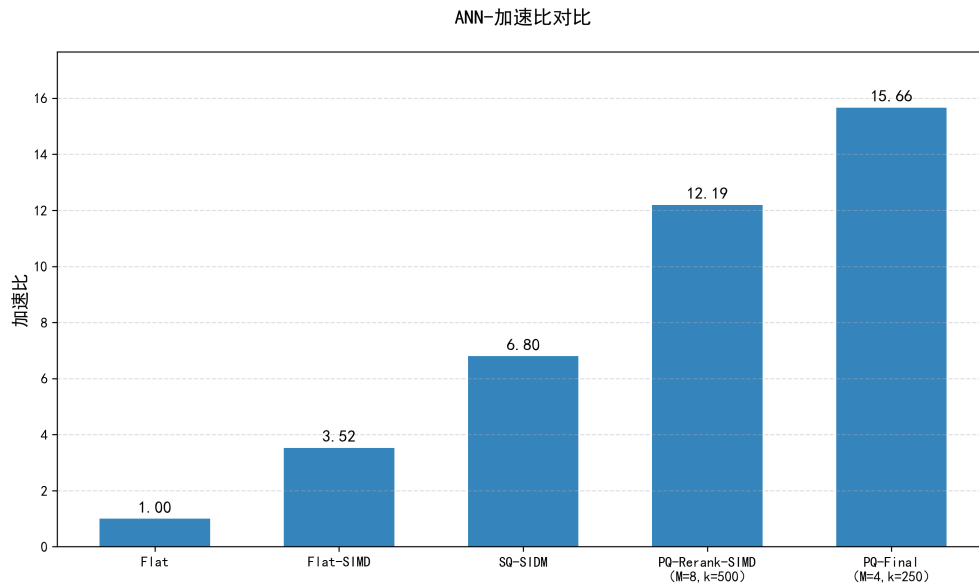


图 2: 加速比对比

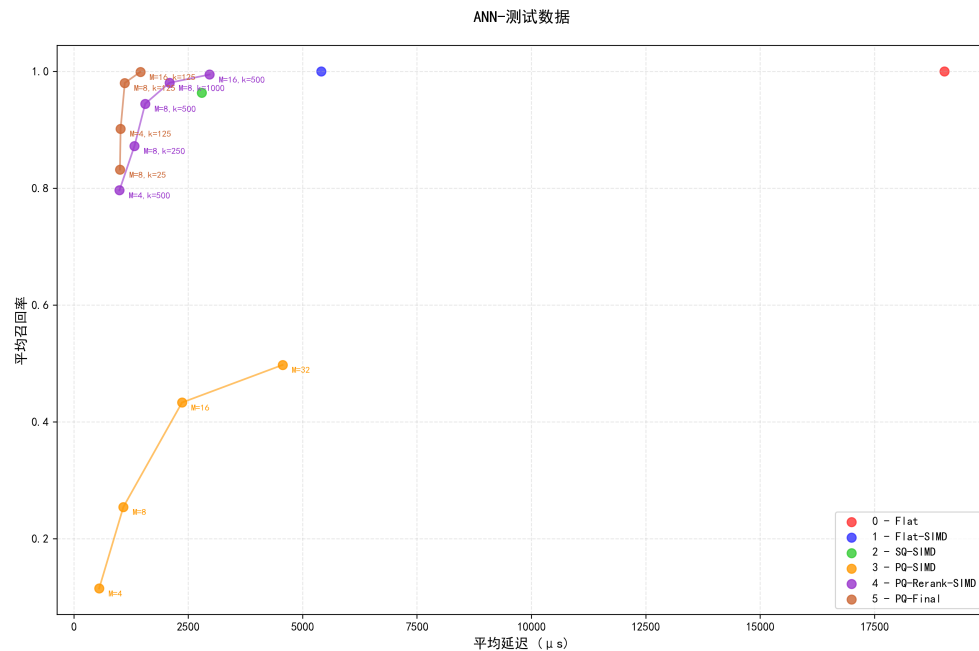


图 3: 测试数据

总体来看，算法的优化效果明显，在保证精度的前提下，最终取得了较高的加速比。观察散

点图可以发现，本实验中由于考察因素较多，在召回率与延迟之间存在一定的 tradeoff，同时各个参数也会造成多重影响，为了进一步分析，接下来将对参数的权衡进行探讨。

(二) 参数权衡

在本实验中，子空间数量 M 与重排深度 rerank_k 的选取共同决定了计算的效率与精度。

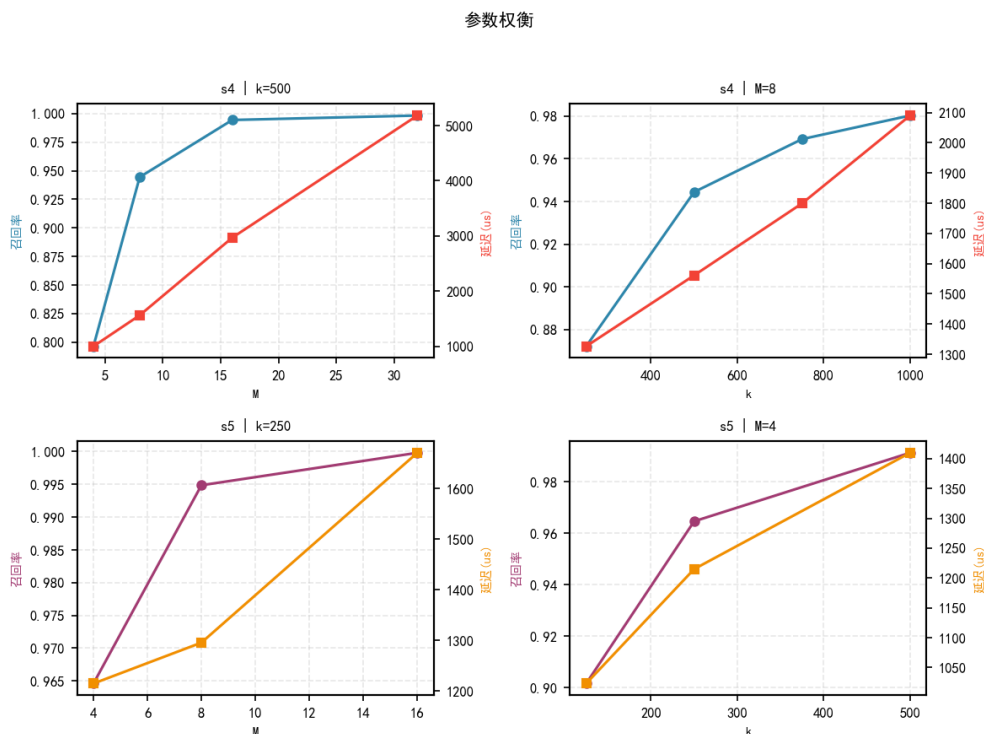


图 4: 参数对比

从数据趋势看， M 决定了“粗排”阶段的精度：在纯 PQ 中，随着 M 增加，召回率虽有提升，但仍低于 0.5，这说明单纯通过增加 M 来提升精度会迅速触及边际效益递减的临界点，因此必须引入 rerank_k 精排来做精度补偿。

重排数 rerank_k 的选取直接影响了量化误差的容忍度。在策略 4 (PQ+Rerank+SIMD) 的算法下，为了将召回率从 0.87 提升至 0.98，rerank_k 必须从 250 扩大到 1000，这导致精排阶段的浮点计算开销剧增，成为延迟的主要来源。实验中也表明，这种大候选集的策略在追求召回率时带来的成本极高，反映出该算法的计算瓶颈。在策略 5 中，采用了局部重排，又通过多方位的优化提升了效率，然而其边际效应依然存在，两个参数之间需要进行平衡与取舍。

碍于时间与计算资源，这里并未为此专门编写参数自适应的额外代码，仅采用手动测试的方式进行粗略预测。经过大量的分析与测试，如果以本次的实验环境带入工程视角，预估策略 5 的 $M=8$, rerank_k=125 是该算法最具性价比的参数。它充分利用了硬件缓存布局和局部精排的多样性，在效率和精度上取得了相对稳定而高效的平衡。

(三) perf 分析

在服务器上进行 perf-stat 分析，选取两个具有代表性的策略与参数进行测试。

- PQ-Rerank-SIMD: $M = 8$, rerank_k = 500;

- PQ-Final: $M = 4$, $\text{rerank_k} = 250$;

表 4: s4-8-500 perf 性能测试结果

缓存性能		指令执行		执行时间	
缓存引用	27,851,323,920	周期数	56,176,666,639	总执行时间	15.50s
缓存缺失	162,143,923 (0.582%)	指令数	156,059,444,321	用户时间	21.69s
		IPC	2.78	系统时间	0.10s

表 5: s5-4-250 perf 性能测试结果

缓存性能		指令执行		执行时间	
缓存引用	30,923,261,623	周期数	53,879,840,078	总执行时间	11.29s
缓存缺失	52,072,542 (0.168%)	指令数	153,101,246,247	用户时间	20.99s
		IPC	2.84	系统时间	3.54s

- 数据局部性: 0.168% 的极低缓存缺失率证明了 SoA 布局对硬件预取的巨大提升, 连续访存内存对齐良好。
- 指令流水线: 2.78/2.84 的 IPC 反映了 SIMD 指令与 NEON 优化带来的较高的指令并行度。高 IPC 表明流水线极少因数据依赖或分支预测失败而停顿, 充分发挥了 ARM 架构的算力。

(四) 问题探讨

在实验与测试的过程中, 也有几个有趣的问题值得探讨。

1. 为什么重排能对 PQ 带来如此大的精度提升?

在这里, 召回一个目标向量是一个复合事件, 召回的概率可以表示为复合概率的形式。PQ 算法虽然精度低, 但在数学上仍然具有有序的统计趋势, 只是因为其自身的量化逻辑, 有大量的向量的 ADC 距离是并列或非常接近的, 此时 Top-K 的筛选在概率上几乎退化为随机采样, 导致误差极大。而重排通过引入原始浮点精度, 将这个条件概率从随机校准为确定的计算, 从而让整体召回率显著提升。

2. 策略 5 的 rerank_k 为什么远低于策略 4?

在这部分, 策略 5 的 rerank_k 实际表示的是每个线程内局部的 rerank_k, 而策略 4 则是全局单独的一个 rerank_k。在不同的环境下, 应该将策略 5 的 rerank_k 乘以调用的线程数才能等效策略 4 的 rerank_k。但是, 由于两者架构差异较大, 实际情况还需考虑局部线程内队列相关的贡献度与概率等问题 (如策略 5 的线程内重排后归并可以弱化局部严重偏差的影响), 该部分主要为数学概率等分析, 这里我们只需知道, 差异的原因在于全局与局部的架构转变即可。

五、 总结与体会

在本次实验中, 我们完成了 ANN 的优化算法与 SIMD 实现, 逐步提升了算法的效率与精度。从暴力算法到 PQ 再到重排, 结合 SIMD 技术, 在实践中掌握了并行设计与算法优化的流程与核心思想。

在一开始，只是使用 SIMD 技术进行简单的并行优化，但其优化效果有限，而后引入了量化的方法来进行压缩，从而进一步提升效率。但当量化带来的近似误差增大，召回率又成了不得不考虑的一个重要因素。为此我们又引入了重排，同时对于重排的策略也进行了相应的思考与改进，最终终于得到了一个高效而精准的算法。

在完成算法后，通过多维度的测试结果进行分析，过程中对于算法参数的权衡以及一些现象的探讨，也让我更能以现实视角考虑并行问题与其背后的深刻意义。此外，在本次实验中由于测试框架的使用，也让我对于服务器指令也有了更多的了解。

总的来说，这场实验让我全方位地了解对于一个问题进行并行优化的完整流程，同时在过程中对于相关原理的学习，也让我收获颇丰。

六、 附录：代码仓库链接

GitHub: <https://github.com/Iriasoul/Parallel-Programming/tree/main/lab02>

NIJUB